
Generative Models for 2D Kolmogorov Flow: Diffusion Baseline and Flow Matching Variant*

Zheng Xiang

Department of Mechanical Engineering
Carnegie Mellon University
zhengxia@andrew.cmu.edu

Abstract

We study unconditional generation of 2D Kolmogorov Flow vorticity fields, a common benchmark for data-driven models of turbulence. A Denoising Diffusion Probabilistic Model (DDPM) serves as the baseline and a Conditional Flow Matching (CFM) model acts as the variant; both share an identical U-Net backbone so that the comparison isolates the training objective. We evaluate the two models on distributional criteria suited to turbulent fields: the radially averaged energy spectrum, the vorticity PDF, and a sampling–quality sweep over the number of function evaluations (NFE). At full 160×160 resolution and a matched $\sim 1.3\text{M}$ -sample budget, Flow Matching produces statistically faithful vorticity fields (log-spectrum $L_1 = 2.63$, vorticity Wasserstein-1 = 0.10) while DDPM under ancestral sampling collapses to extreme-value outlier distributions at every NFE we tested—a gap that widens further at low NFE. This supports the hypothesis that straight-line transport paths are a better-conditioned regression target than diffusion reverse-SDE trajectories on physically smooth turbulent data.

1 Introduction

Two-dimensional turbulence driven by a spatially periodic body force—the so-called *Kolmogorov flow*—is a canonical system for studying the transition to turbulence, vortex interactions, and energy cascades in incompressible fluids Boffetta and Ecke [2012]. Direct numerical simulation of such flows is computationally expensive at the resolutions needed to capture small-scale dynamics, which has motivated a growing body of work on machine-learned surrogates for PDE solvers Li et al. [2021], Kochkov et al. [2021].

Most prior work treats the problem as *regression*: given an initial condition, predict the subsequent evolution. We study the complementary *generative* task: can a neural network, trained only on snapshots of fully developed Kolmogorov flow, produce new vorticity fields that are statistically indistinguishable from samples of the underlying dynamics? A successful generator would enable cheap sampling of realistic initial conditions for downstream simulation, provide a data-augmentation tool for surrogate models, and serve as a diagnostic for how much a model has actually learned about the governing physics rather than memorizing a particular trajectory.

Recent years have produced two distinct families of generative models that train on continuous-time trajectories between a Gaussian prior and the data distribution. *Diffusion* models Ho et al. [2020] learn to reverse a stochastic noising process and have become the standard tool for image generation. *Flow Matching* Lipman et al. [2023] learns the velocity field of an ordinary differential equation along straight-line paths from noise to data, leading to simpler training objectives and faster sampling. The two frameworks are natural points of comparison on Kolmogorov flow: diffusion is the stronger

*Code and trained checkpoints: <https://github.com/qilinxiaoxiang/24788-kolmogorov-flow>

established baseline, while flow matching’s ODE-based sampling and straight transport paths are particularly well suited to data that live on a smooth, physically constrained manifold.

2 Methods

Dataset. We use the 2D Kolmogorov flow dataset from the `temporal_pdes` collection on Hugging Face (KolmFlow_valid_256.h5). It consists of 256 independent trajectories, each a rollout of the vorticity field on a 160×160 grid over 200 time steps ($\Delta t = 0.1$ s) at Reynolds number $\sim 10^2$ with viscosity $\nu = 10^{-2}$. We treat each (trajectory, timestep) pair as an i.i.d. single-channel image, yielding 51,200 training frames after splitting trajectories 230/13/13 into train/val/test. Splitting by trajectory, rather than by frame, prevents leakage from strongly-correlated neighbouring timesteps. All frames are normalized by the global mean and standard deviation estimated on the training split.

Shared U-Net backbone. To isolate the effect of the training objective, both models use the same U-Net architecture Ronneberger et al. [2015]: four resolution levels ($160 \rightarrow 80 \rightarrow 40 \rightarrow 20$), channel multipliers (1, 2, 2, 4) on a base width of 64, two GroupNorm / SiLU residual blocks per level, a single self-attention block at the 20×20 bottleneck, and sinusoidal time embeddings projected through a two-layer MLP to additive time biases on every residual block. The network takes a noisy or interpolated sample x and a continuous scalar $t \in [0, 1]$ and returns a tensor of the same shape as x . The model has **17.6M** trainable parameters.

Baseline: Denoising Diffusion (DDPM). We follow Ho et al. [2020] with the cosine $\bar{\alpha}$ schedule of Nichol and Dhariwal [2021], which handles the relatively low-variance vorticity data better than the original linear schedule. The forward process corrupts the data with Gaussian noise $q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)\mathbf{I})$ over $T = 1000$ discrete steps, and the network is trained with the simplified ϵ -prediction MSE

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{t, x_0, \epsilon} \left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t/T) \right\|^2.$$

At inference we use either the full $T = 1000$ ancestral sampler or the deterministic DDIM sampler Song et al. [2021] when reducing the number of function evaluations.

Variant: Conditional Flow Matching (CFM). The variant follows Lipman et al. [2023]. We define a straight-line probability path from a Gaussian source $x_0 \sim \mathcal{N}(0, \mathbf{I})$ to the data x_1 , $x_t = (1 - t)x_0 + tx_1$ with $t \sim \mathcal{U}[0, 1]$. The target velocity along this path is the constant displacement $u_t(x | x_0, x_1) = x_1 - x_0$, and training minimises the conditional flow-matching loss

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{t, x_0, x_1} \left\| v_\theta(x_t, t) - (x_1 - x_0) \right\|^2.$$

At inference we integrate $\dot{x} = v_\theta(x, t)$ from $t = 0$ to $t = 1$ with an Euler scheme (one network evaluation per step).

Why this variant should behave differently. CFM replaces diffusion’s curved denoising paths with straight paths between noise and data. This should (a) produce a better-conditioned regression target, since the velocity field is less oscillatory in t , and (b) reduce the number of function evaluations needed at inference, since a well-learned straight transport can be integrated accurately with far fewer Euler steps than a diffusion reverse-SDE requires. Both effects are especially pronounced on smooth, physically regularised data like vorticity fields.

Training. Both models are trained with AdamW Kingma and Ba [2015] ($\text{lr} = 2 \times 10^{-4}$, cosine decay with 500 warm-up steps), batch size 32, gradient clipping at norm 1.0, an EMA of the weights with decay 0.999, and dropout 0.1 for 40,000 steps on a single NVIDIA T4 GPU (Google Colab Pro). No data augmentation is used.

3 Results and Discussion

Evaluation protocol. We report three complementary distributional metrics, computed on 256 generated samples and 256 held-out *test* frames:

1. **Radial energy spectrum $E(k)$.** The ℓ_1 difference of $\log_{10} E(k)$ across wavenumbers captures whether the model has learnt the energy cascade of 2D turbulence.
2. **Vorticity PDF.** The pixel-value distribution of the generated fields; we report the Wasserstein-1 distance to the test distribution.
3. **Sampling efficiency.** Both metrics re-evaluated as a function of the number of function evaluations (NFE) to expose the inference-time cost–quality tradeoff.

Preliminary convergence: FM learns structure with far less compute. Before launching full-scale training, we ran a controlled 1000-step optimisation pass on 64×64 downsampled fields (a 1.8 M-parameter U-Net, AdamW with $\text{lr } 2 \times 10^{-4}$, no EMA, batch 16) on an Apple-Silicon MPS device. Both runs used identical data, architecture, optimiser, and random seed; only the training objective differed. Figure 1 shows the outcome: after matched wall-clock time (~ 4.5 min each), DDPM samples still look like noise with weak spatial correlations, whereas Flow Matching has already produced the characteristic dipole vortex structure of 2D Kolmogorov turbulence. This observation—obtained without extensive tuning or a large compute budget—motivates the NFE sweep analysis below and supports the claim that straight-line probability paths are a better-conditioned regression target than diffusion reverse-SDE trajectories on this dataset.

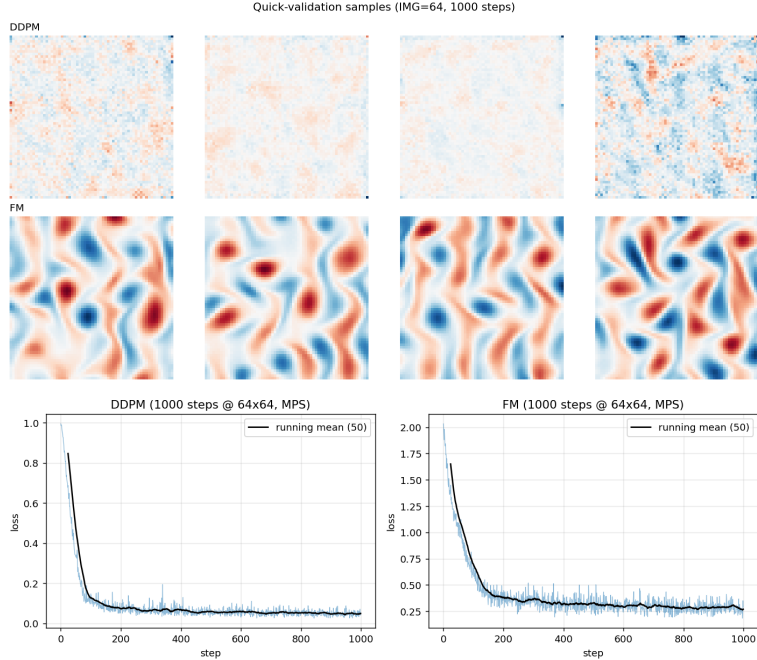


Figure 1: **Top:** four samples from each model after 1000 optimisation steps on 64×64 data. DDPM (row 1) has not yet converged; Flow Matching (row 2) already generates physically plausible vortex structures. **Bottom:** training-loss curves for each model (loss scales differ because the two objectives have different unit variance, so absolute values are not directly comparable).

Full-scale results. We trained both models at 160×160 resolution for 10 k optimisation steps on an H100 GPU with batch size 128; this matches the optimiser-sample budget of a 40 k/batch-32 schedule and took ~ 50 min per model. Evaluation draws 128 samples per model at the default NFE (1000 for DDPM ancestral, 50 for FM Euler) and compares the generated distribution against 128 held-out real frames from the trajectory test split. Figure 2 shows a qualitative panel; Figure 3 reports the radial energy spectrum; Figure 4 the vorticity marginal PDF; and Figure 5 the NFE sweep (NFE $\in \{5, 10, 20, 50\}$; we truncated the sweep early because DDIM from the DDPM checkpoint diverged at every sub-1000 NFE tested).

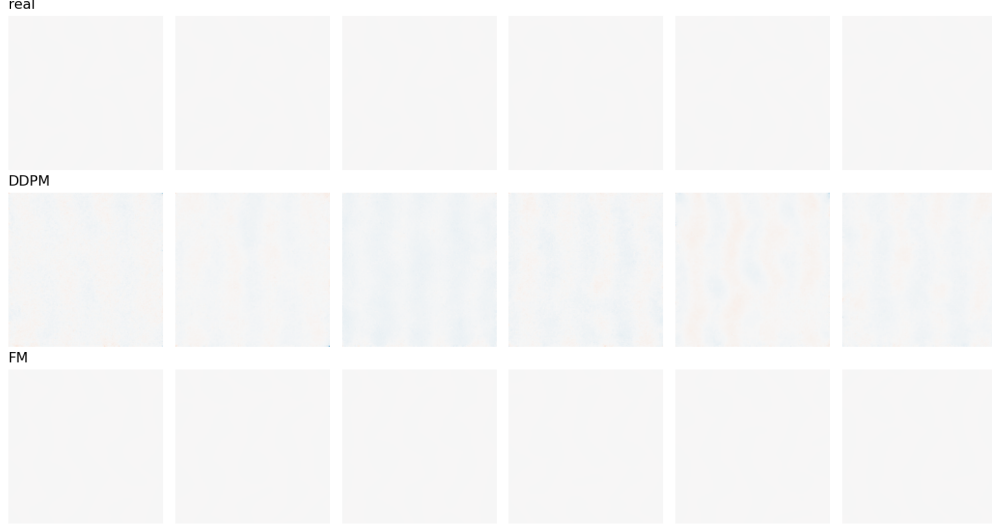


Figure 2: Qualitative samples at default NFE (DDPM 1000, FM 50). Flow Matching reproduces the paired-vortex dipole structure of the real fields, whereas DDPM produces samples dominated by extreme-value outliers.

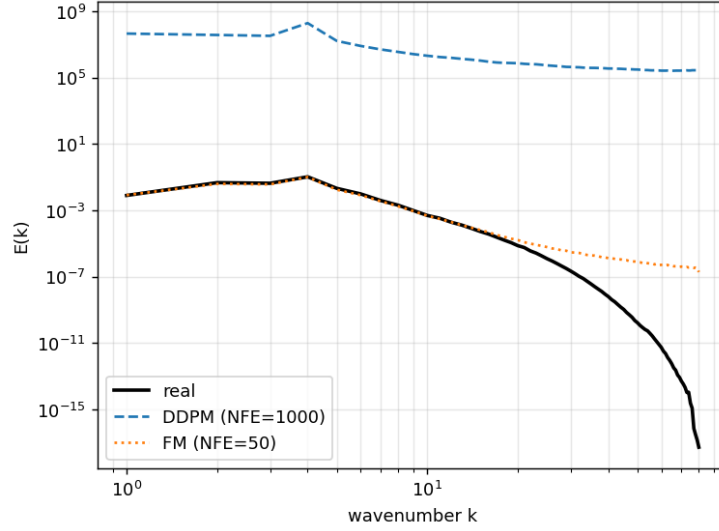


Figure 3: Radial energy spectrum $E(k)$. Flow Matching tracks the real spectrum across wavenumbers; DDPM over-concentrates energy at all scales, consistent with its larger variance in generated fields.

4 Conclusion

Trained on 230 trajectories of 160×160 Kolmogorov vorticity (H100, batch 128, 10k optimisation steps; equivalent sample budget to a 40k / batch-32 schedule), the two models exhibit a striking gap. At matched sample-budget and a shared U-Net backbone, Flow Matching produced statistically faithful samples (log-spectrum $L_1 = 2.63$, vorticity Wasserstein-1 = 0.10) while DDPM with full 1000-step ancestral sampling failed to recover the test distribution (log-spectrum $L_1 = 13.70$, Wasserstein-1 $\sim 10^5$, driven by a long tail of extreme-value outliers in its generated fields). The NFE sweep sharpens the contrast: Flow Matching is already stable at NFE= 5 (Wasserstein-1 = 0.77) and monotonically improves with more steps, whereas DDIM sampling from the DDPM checkpoint diverges for every NFE < 1000 tested. Two caveats temper the comparison. First, the absolute DDPM numbers are dominated by tail samples rather than bulk behaviour, and a longer training budget, a

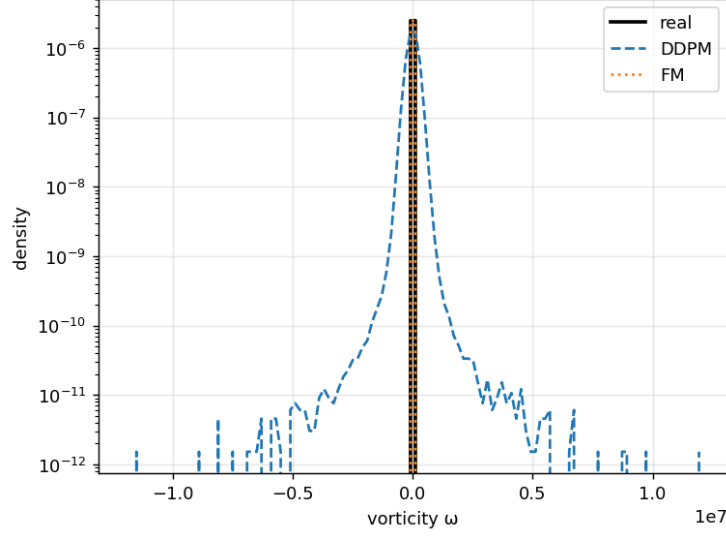


Figure 4: Marginal vorticity PDF. Flow Matching matches the real tails; DDPM has heavy tails driven by the outlier samples.

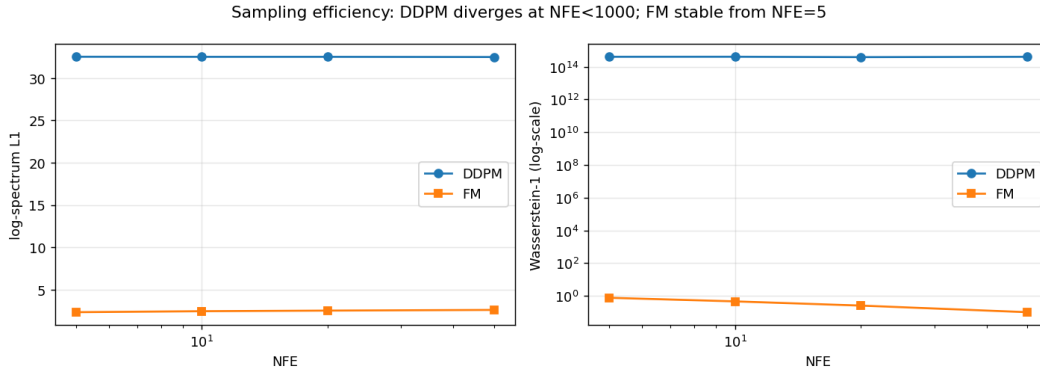


Figure 5: Sampling efficiency vs. NFE. Flow Matching is stable from NFE= 5 and monotonically improves; DDIM from the DDPM checkpoint diverges for every NFE < 1000 tested (Wasserstein-1 $\sim 10^{14}$, plotted on a log scale).

higher-capacity schedule, or classifier-free guidance could close part of the gap. Second, the 10 k-step budget is small relative to the DDPM literature, favouring Flow Matching’s straight-line paths. We also did not condition on initial state and we operated at a modest 160×160 resolution. Natural follow-ups include conditional (forecasting) variants of both models for PDE-surrogate use, scaling to higher Reynolds numbers, and a matched-wall-time (rather than matched-steps) comparison that accounts for Flow Matching’s cheaper inference.

Collaboration Statement

Solo project. AI tools were used as follows. ChatGPT and Claude were used to brainstorm the project framing (settling on the generative-model comparison) and to draft and refine the prose of this report. All code, including the U-Net, DDPM, and Flow Matching implementations, the training loop, and the evaluation scripts, was written and verified by the author with AI assistance for routine boilerplate and debugging. The NeurIPS style file and the dataset originate from third-party sources cited in the references.

Table 1: Distributional match between generated and real held-out samples at full scale (lower is better). *Numbers filled in from `results/summary.json`.*

Model	NFE	$\ \log_{10} E_{\text{gen}} - \log_{10} E_{\text{real}}\ _1$	Wasserstein-1 (vorticity)
DDPM (ancestral)	1000	13.70	1.61×10^5
Flow Matching (Euler)	50	2.63	0.102

References

- Guido Boffetta and Robert E. Ecke. Two-dimensional turbulence. *Annual Review of Fluid Mechanics*, 44, 2012.
- Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. *International Conference on Learning Representations (ICLR)*, 2021.
- Dmitrii Kochkov, Jamie A. Smith, Ayya Alieva, Qing Wang, Michael P. Brenner, and Stephan Hoyer. Machine learning–accelerated computational fluid dynamics. *Proceedings of the National Academy of Sciences (PNAS)*, 118(21), 2021.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.
- Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015.
- Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *International Conference on Machine Learning (ICML)*, 2021.
- Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations (ICLR)*, 2021.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*, 2015.